template_pitch_deck: Reproducible, Validated Pitch-Deck Generation

Short/Medium/Long PDF+PPTX decks from one token-resolved content source

Daniel Ari Friedman
Active Inference Institute
daniel@activeinference.institute
ORCID: 0000-0001-6232-9096
DOI: 10.5281/zenodo.21281509

2026-07-08

Contents

1 Abstract

Research groups routinely need to pitch their work — to funders, partners, or collaborators — yet pitch decks are almost never treated as reproducible research artifacts: they are hand-assembled in proprietary slide tools, contain unverifiable claims, and cannot be regenerated when the underlying facts change. `template_pitch_deck` closes that gap. It generates six artifacts from one token-resolved content source — short, medium, and long decks, each in both PDF and PPTX — with every numeric claim traced back to a live introspection of the repository it describes, every `{{TOKEN}}` substitution verified to have actually landed, and every sentence checked against a denylist of pitch-deck clichés.

The flagship content pitches `template_template`, this monorepo's own autopoietic meta-project, to a meta-science and science-integrity audience — the kind of pitch an organization like the Active Inference Institute or COGSEC would actually hand to a funder. The rendering engine itself is new, reusable infrastructure: `infrastructure/rendering/slide_deck.py` (ReportLab, PDF) and `infrastructure/rendering/pptx_deck.py` (python-pptx) both consume the same `DeckContent` model, so a PDF and a PPTX built from identical content carry identical slide counts and identical text — verified by direct read-back of both file formats, not by inspection.

Keywords: pitch deck, slide generation, reproducible research communication, meta-science infrastructure, token validation, PPTX, PDF rendering.

2 Introduction

Every other public exemplar in this monorepo renders a manuscript: prose, figures, citations, a combined PDF. None of them render a *pitch* — a short-form, persuasion-oriented artifact whose job is not to document a method but to move an audience to a decision. Pitch decks live outside the repository's reproducibility guarantees entirely: built by hand in proprietary tools, their claims untethered from any generator, their content unchecked for the boilerplate language ("synergy," "disruptive," "10x") that makes a real pitch land badly with a sophisticated audience.

`template_pitch_deck` treats a pitch deck the same way this repository treats a manuscript: as a build artifact with a single source of truth, a validation gate, and a reproducibility guarantee. Three properties make that possible.

One content source, six artifacts. `manuscript/deck_content_{short,medium,long}.yaml` define the slide-by-slide narrative at three lengths; `manuscript/deck_tokens.yaml` supplies the facts. `scripts/render_decks.py` resolves tokens once and calls both renderers — `infrastructure.rendering.slide_deck.render_pdf` and `infrastructure.rendering.pptx_deck.render_pptx` — against the identical resolved content, so PDF and PPTX cannot drift from each other.

Facts, not fabrication. Every numeric claim in `deck_tokens.yaml` about the pitch subject (`template_template`) is generated from a live read of that project's own `README.md`/`AGENTS.md` or the repository's public exemplar roster — never hand-typed, never a plausible-sounding invented metric. `src/token_resolution.py` raises loudly if any `{{TOKEN}}` in the content source has no corresponding resolved value, and a rendered artifact's extracted text is checked to contain zero leftover `{{` literals.

Cliché is a lint, not a vibe. `src/cliche_lint.py` runs a word-boundary denylist of pitch-deck stock phrases over every resolved slide before render. A pitch that reads as generic — regardless of how factually accurate it is — fails the same way an uncovered line of code fails a coverage gate.

The remainder of this manuscript covers the deck-rendering architecture (sec. 3), the validation model (sec. 4), and the reproducibility guarantees (sec. 5) that let `uv run python scripts/render_decks.py --project templates/template_pitch_deck` produce the same six files, byte-for-byte, on any machine with this repo checked out.

3 Deck Rendering Architecture

3.1 Content model

`infrastructure/rendering/slide_deck.py` defines the format-agnostic content model shared by both renderers:

- `Slide` — one slide's title, bullets, an optional speaker-notes string, an optional local figure path, and a `kind` (`title`, `section`, or `content`).
- `DeckContent` — a deck title/subtitle plus an ordered tuple of `Slide`.
- `SlideBudget` — the three published lengths (`SHORT` $\leq$ 10 slides, `MEDIUM` $\leq$ 22, `LONG` $\leq$ 45) and `filter_deck_for_budget`, a pure function that truncates a deck to a budget without mutating slide order or content.

3.2 Two renderers, one model

`infrastructure/rendering/slide_deck.render_pdf` draws each slide directly with ReportLab's canvas API onto a 16:9 page — no LaTeX/pandoc dependency, following the same "project needs pixel-level layout control" reasoning that led `template_newspaper` to a hand-written ReportLab engine, but implemented as a generic, project-agnostic *format* renderer (like `infrastructure/rendering/docx_renderer.py` or `epub_renderer.py`) rather than a project-owned layout engine, because a slide deck — unlike a newspaper's column geometry — is a reusable output shape any future project can consume.

`infrastructure/rendering/pptx_deck.render_pptx` builds the identical slide sequence with `python-pptx` (an opt-in dependency: `uv sync --group rendering-pptx`), matching title/section/content slide handling 1:1 with the PDF path. Both renderers are exercised by `tests/infra_tests/rendering/test_slide_deck.py` and `test_pptx_deck.py`, including a direct parity test that renders the same `DeckContent` through both paths and asserts the PDF page count equals the PPTX slide count.

3.3 Project-side content

`projects/templates/template_pitch_deck/src/` holds only pitch-deck-domain code — loading `manuscript/deck_content_*.yaml` into `DeckContent` objects, resolving `{{TOKEN}}` values, and linting for cliché — never layout/drawing code. `scripts/render_decks.py` is the thin orchestrator that ties content loading, token resolution, cliché linting, and the two infra renderers together into the six published artifacts under `output/{pdf,pptx}/`.

4 Content and Validation

4.1 The flagship pitch

The shipped content pitches `template_template` — this monorepo's autopoietic meta-project, which introspects the repository's own architecture and regenerates its manuscript from live counts — to a meta-science and science-integrity audience. Every length follows the same arc: the problem (research communication and reproducibility are under-tooled), the solution (a two-layer, thin-orchestrator monorepo with a 90%/60% coverage floor, no-mocks testing, and multi-platform publication), proof (real, currently-measured facts: exemplar count, coverage floors, the publishing surface `template_template` itself already reaches), and an ask. Medium and long variants add landscape, architecture, and roadmap detail; long adds a full governance/confidentiality walkthrough and an appendix.

4.2 Token resolution

`src/token_resolution.py` implements the same `{{TOKEN}}` convention used by `infrastructure/rendering/manuscript_injection.py` (`\{\{[A-Z][A-Z0-9_]*\}\}`), scoped to `manuscript/deck_content_*.yaml` instead of `manuscript/*.md`. `resolve_tokens` raises if any token in the content has no matching key in `manuscript/deck_tokens.yaml` — mirroring `template_madlib`'s `test_all_manuscript_tokens_are_generated` pre-substitution coverage check, adapted to deck content. `scripts/audit_deck_content.py` runs this check plus the cliché lint in one pass and exits non-zero on either class of failure; both failure modes are proven to actually fire (a deliberately-broken fixture triggers each) before the real content is checked clean.

4.3 Cliché lint

`src/cliche_lint.py` maintains a word-boundary-safe denylist of pitch-deck stock phrases ("synergy," "disrupt," "10x," "game-changing," "rocket ship," "paradigm shift," and more). It is checked against every resolved slide across all three lengths as part of the same audit script, and — like the token check — is proven to fire on a deliberately cliché-laden sentence before the real content's cleanliness is trusted.

5 Reproducibility

`scripts/render_decks.py` is deterministic: given the same repository state (`manuscript/deck_content_*.yaml`, `manuscript/deck_tokens.yaml`, and the live facts `src/deck_tokens.py` reads from `template_template`'s own files and the public exemplar roster), two consecutive runs produce byte-identical PDF and PPTX output. No wall-clock timestamps appear in slide content; the only place a generation time is recorded is deck metadata, following the `STEGANOGRAPHY_DETERMINISTIC`-style convention already used elsewhere in this repository for reproducible builds.

```
uv run python scripts/render_decks.py
# → output/pdf/template_template_pitch_{short,medium,long}.pdf
# → output/pptx/template_template_pitch_{short,medium,long}.pptx

uv run pytest projects/templates/template_pitch_deck/tests/ \
  --cov=projects/templates/template_pitch_deck/src --cov-fail-under=90

uv run python scripts/audit/check_template_drift.py
```

This project participates in the standard multi-project pipeline like any other public exemplar (`./run.sh --project templates/template_pitch_deck --pipeline --core-only`) and requires no LLM/Ollama stage — deck content is authored and token-resolved, not generated at render time by a model call, which keeps the artifact deterministic and the core pipeline Ollama-optional.

References